

PITSENSE

Project Manual — Features, Architecture, Files, Testing & Hackathon Readiness

Current verified baseline: 53/53 backend tests passing.

PitSense is a motorsport engineering assistant combining team-radio speech, audio perception, telemetry, temporal analysis and an engineering decision layer.

1. Ready-to-send group message

Copy/paste:

■ **PitSense is basically hackathon-ready.**

We've finished the core backend and the current build is **53/53 automated tests passing**.

What it does:

- Team-radio audio → text using Parakeet ASR
- Vocal emotion + acoustic speech analysis
- 0–100 Stress Index from audio, speech and transcript signals
- Radio → telemetry/lap matching
- Multi-lap temporal stress/performance analysis
- Engineering decisions like NO ACTION, MONITOR, RADIO INTERVENTION and PIT & INSPECT
- SQLite session persistence across backend restarts
- Dashboard with telemetry/trend/graph data
- Explicit data-quality and fallback handling

Architecture: FastAPI → audio normalisation → ASR/perception → stress engine → telemetry + temporal analysis → decision engine → persistence/dashboard.

The main things left are **speed/responsiveness**, **multi-user/session isolation**, and **final UI/demo polish**. So we're in optimisation + demo-hardening mode rather than adding random features. ■

2. Architecture

Layer	Responsibility	Main components
Input/API	Audio upload, simulation and response contract	FastAPI / app.py
Audio	Normalise arbitrary audio for models	FFmpeg → 16 kHz mono WAV
ASR	Transcribe radio	Parakeet TDT 0.6B v3
Vocal perception	Emotion probabilities	Wav2Vec2 speech-emotion model
Acoustic perception	Speech-level measurements	RMS, ZCR, non-silence ratio
Stress	Fuse signals into 0–100 index	stress_engine.py
Telemetry	Radio/lap/performance context	OpenF1 pipeline + dataset loader
Temporal	Analyse changes across observations	temporal_analysis.py
Decision	Recommend engineering action	decision_engine.py
Persistence	Store session observations/history	SQLite / SessionManager
Presentation	Dashboard, graphs and API data	Frontend + API response contract

Flow: Audio → FFmpeg → Parakeet + emotion + acoustic features → Stress Index → telemetry context → temporal analysis → engineering decision → dashboard + SQLite history.

3. Feature inventory

- Team-radio transcription with Parakeet.
- Local audio emotion classification with top label, confidence and per-class probabilities.
- Acoustic speech features: RMS, zero-crossing rate and non-silence ratio.
- Stress Index using vocal (0.50), speech (0.20) and transcript (0.30) weights.
- Stress states: CALM, ELEVATED, STRESSED and CRITICAL.
- Transcript urgency cues split into critical and elevated phrases.
- Telemetry/radio-to-lap interval matching with unavailable handling.
- Temporal analysis: stress trend, performance direction, sample count and correlation when sufficient data exists.
- Engineering decision layer: NO_ACTION, MONITOR/monitor-performance paths, RADIO_INTERVENTION and PIT_AND_INSPECT.
- Four explicit data-quality states: AVAILABLE, PARTIAL, INSUFFICIENT and UNAVAILABLE.
- SQLite persistence and restart restoration.
- Simulation mode using the frozen dataset.
- Gemini failure fallback to local/deterministic synthesis.
- Automated backend test suite: 53 tests, all passing.

4. Important AI modules

File	What it handles
backend/ai/asr_model.py	Loads Parakeet; FFmpeg conversion; transcription; combined perception path.
backend/ai/audio_emotion.py	Loads HF audio-emotion model; returns label/confidence/probabilities; explicit unavailable sentinel.
backend/ai/stress_engine.py	Computes vocal, speech and transcript scores; combines them; assigns stress state.
backend/ai/temporal_analysis.py	Multi-observation trends, performance direction, sample sufficiency and correlation.
backend/ai/decision_engine.py	Turns driver state, stress, urgency, temporal evidence and issues into engineering decisions.

5. Dataset + telemetry

File	Responsibility
dataset/download_dataset.py	OpenF1 session/radio discovery, audio download, radio→lap matching and metadata generation.
dataset/metadata.csv	Frozen dataset metadata and telemetry linkage.
dataset/audio/	Frozen audio samples used for simulation/tests.
backend/dataset_loader.py	Loads metadata, simulation samples and telemetry for audio files.

Latest reported dataset build: target 250 rows; final run produced 250 dataset rows, including 100 downloaded audio files, 65 telemetry-linked samples and 35 radio-only samples. The team decided this dataset size is sufficient for the hackathon.

6. Persistence + backend

Component	Role
backend/app.py	FastAPI application, upload/simulation integration and dashboard response schema.
SQLite / SessionManager	Durable observations, active-session identity, restart restoration and reset operations.
backend/test_phase9_persistence.py	Verifies the complete persistence lifecycle including simulated restart.
backend/test_phase10_hardening.py	Health endpoint, schema contract, fallback paths, missing audio and data-quality states.

7. Testing

Full command:

```
.\backend\.venv\Scripts\python.exe -m pytest backend -v
```

Suite	Passing
Dataset pipeline	8
Phase 10 hardening	6
Phase 7–8 temporal/decision	10
Phase 9 persistence	13
Phase 9 simulation	11
Telemetry	2
Telemetry contract	3
TOTAL	53 / 53

Latest run: 53 passed, 1 warning, 0 failures, in 39.39 seconds. The warning is a Starlette/httpx TestClient deprecation warning; it is non-failing.

8. File/test map

	Purpose	
dataset_pipeline.py	Dataset parsing, radio/lap matching, metadata preservation, simulation endpoint, deduplication and frozen audio validation.	
phase7_8.py	Temporal-analysis and decision-engine unit tests plus full /upload integration.	
phase9_persistence.py	SQLite lifecycle, restart restoration, continuation and reset.	
phase9_simulation.py	Simulation discovery, audio serving, upload and edge cases.	
phase10_hardening.py	Health/schema/fallback/404/data-quality hardening.	
telemetry.py	Dataset validation and real lap_04 telemetry upload.	

	Purpose	
_telemetry_contract.py	Telemetry-series availability/insufficiency/unavailable contract.	
uirements.txt	Backend Python dependencies.	
d	.gitignore	Project documentation / repository hygiene.

9. What to improve next

Priority	Improvement	Reason
1	Inference speed	Local ASR + emotion inference is the main UX bottleneck.
2	Multi-user sessions	Shared deployment should isolate each friend's history/session.
3	Dashboard polish	Loading states, responsive graphs and clearer transitions make the demo feel production-ready.
4	Shared deployment	A GPU-capable machine can host models, SQLite and dataset behind one browser URL.

Speed optimisation checklist

- Keep ASR/emotion models loaded once at process startup.
- Profile FFmpeg, ASR, emotion, feature extraction and response latency separately.
- Reuse the single converted WAV across perception stages (already done in transcribe_and_perceive).
- Run independent post-ASR perception work in parallel where safe.
- Cache inference results for repeated frozen-dataset demo samples.
- Benchmark GPU acceleration/precision options on the RTX 5050 machine before changing the production path.
- Use a lightweight interactive inference path where appropriate, while retaining full long-form transcription for complete samples.

10. Sharing PitSense with friends

For a full shared demo, run the complete backend/frontend/models on one GPU-capable machine. Friends should access the browser UI; they should never access the SQLite file directly.

Recommended temporary hackathon setup: RTX 5050 laptop as server → FastAPI + models + frozen dataset + SQLite → secure tunnel → friends/judges in browsers.

Keep **.env**, **backend/data/** and **uploads/** out of Git. For multiple simultaneous users, introduce a per-browser/session ID before opening the instance widely.

11. Demo sequence

- Explain the problem: radio, voice and telemetry each tell only part of the story.
- Upload/play a real sample and show the transcript.
- Show emotion/acoustic perception feeding the Stress Index.
- Show telemetry and lap context.
- Use multiple observations to demonstrate the graph/trend.

- Trigger or show an engineering decision and explain its evidence.
- Demonstrate persisted session history.
- End with the three honest next steps: speed, multi-user isolation and UI polish.

12. Current limitations

- Local model inference can be slow, especially on lower-end hardware.
- Audio emotion is an engineering-support signal, not a clinical/psychological diagnosis.
- Transcript cues are intentionally simple keyword logic.
- Correlation is not fabricated when there are too few observations.
- The current application is best treated as a hackathon single-instance baseline until session isolation is added.

13. Release checklist

- Run full pytest suite and require 53/53 (or a consciously updated baseline).
- Check git status before pushing.
- Never commit API keys or .env.
- Keep a known-good frozen commit for the 16 August round.
- Prefer optimisation and reliability changes over large architectural rewrites immediately before the event.
- Rehearse the demo on the exact machine/network that will be used.

14. One-page technical summary

Input: audio/team radio. **Normalisation:** FFmpeg 16 kHz mono WAV. **Perception:** Parakeet ASR + Wav2Vec2 emotion + RMS/ZCR/non-silence. **Fusion:** 0–100 Stress Index. **Context:** telemetry + lap matching. **Temporal:** trends/performance/correlation. **Decision:** engineering action/severity. **Persistence:** SQLite. **Presentation:** FastAPI-backed dashboard. **Validation:** 53/53 backend tests passing.

Bottom line: The core PitSense architecture is sufficiently complete for the hackathon baseline. The highest-value remaining work is making the existing system faster, smoother for shared use and more polished in the live demo.